A Comparability Protocol for Benchmarking Relational Database Access Layers in Express.js

Mateusz Miotk[a,*]

[a] *Faculty of Mathematics, Physics and Informatics, University of Gdańsk, Wita Stwosza 57, 80-308, Gdańsk, Poland*

Abstract

Context: Relational access-layer benchmarks can compare unequal results or report latency at unequal proximity to saturation. **Objectives:** We develop and evaluate a protocol for deciding when configured implementations are comparable and what conclusions their measurements support. **Methods:** The protocol requires task-derived read checks, cross-implementation equivalence, post-write state validation, a predeclared configuration-selection policy, and separate capacity and latency operating points. An open Express.js case study compares nine policy-selected configurations and two tuned native references on PostgreSQL, MySQL, and five access patterns in a 25-run randomized-block campaign. A second same-host campaign tests selected rank reversals; a source-located audit applies the protocol to nine external benchmarks. **Results:** The largest observed spread is the tested deep fetch: 7.01× on PostgreSQL and 5.12× on MySQL. With common SQL executed through raw facilities, the spreads are 1.71× and 2.03×; this compound contrast isolates no single mechanism. Deep-fetch p99 contracts from 18–108 to 2.3–4.9 ms on PostgreSQL and from 28–121 to 1.9–5.5 ms on MySQL at the stable 50% capacity target. Four material cross-stack rank reversals recur in both same-host campaigns; close MySQL insert ranks do not. **Conclusion:** The protocol provides an auditable discipline for establishing and reporting comparability, rather than a claim of universal

*Corresponding author.
Email address: mateusz.miotk@ug.edu.pl (Mateusz Miotk)

rankings or library-intrinsic overhead. Its external audit has one rater and the adapters lack independent human review, so independent applicability remains to be tested.
Keywords: benchmarking methodology, database access layer, object-relational mapping, Node.js, response-time tail (p99), empirical software engineering

---

1. Introduction

Express.js is among the most widely used web frameworks for Node.js, and almost every non-trivial service built on it reaches a relational database through some *access layer*. Access layers trade raw performance for ergonomics, compile-time type safety, and protection from pitfalls such as the N+1 query problem, where fetching a result graph issues one query per parent row instead of one for the result graph [1] — a pervasive anti-pattern (Section 2). Native drivers expose the wire protocol directly (`pg`, `mysql2`); query builders assemble SQL programmatically (Knex); and object-relational mappers (ORMs) map rows to objects, following patterns such as Data Mapper and Active Record [2]. Node.js back-end performance under load is a recognized empirical topic [3, 4, 5], but that literature varies the runtime and framework, not the access layer.

The most visible comparisons are vendor-authored. Peer-reviewed studies in the sources searched cover up to three ORMs in a controlled layer comparison; a broader Node-Bench study includes five access products but also changes frameworks and environments, while the dual-engine precedent holds the access layer fixed [6, 7, 8, 9, 10, 11]. Table 1 records these coverage limits. The sole broad suite reporting a tail gives one P95 at one load point [12], and the academic studies time inside the process rather than through an HTTP framework [10, 13]. These gaps motivate the case-study coverage, not a priority claim. A more basic gap is methodological: the reported numbers are not established to be *comparable* — a layer returning fewer fields or erroring faster than it works can look faster, a library's chosen query strategy is conflated with its own cost, and a

single, arbitrary load point conflates capacity, tail latency under demand, and per-request latency. Establishing that comparability is the problem this paper takes up.

This paper addresses these gaps with a *comparability protocol* and a non-vendor-authored case study that applies it (Section 3 defines the term): an *identical* Express application served through all nine access layers (two native drivers, Knex, and six ORMs: Drizzle, Prisma, Sequelize, TypeORM, Objection, MikroORM) over five representative access patterns — point read, keyset range scan, deep fetch, per-author aggregation, and insert — rather than a production workload. Seven layers run against *both* engines under an identical schema, dataset, and pool size; two tuned native-driver baselines provide disclosed named-preparation and binary-protocol reference points rather than an optimization ceiling. Every (layer, engine, pattern) reports throughput (req/s) and the *high-load response-time p99 under equal closed-loop demand* at the fixed 50-connection operating point, over 25 repeated runs. At this near-saturation point the p99 largely tracks capacity and queueing, so a lower-capacity layer looks worse on the tail even when its sub-saturation latency is not; we therefore also report the tail at *matched utilization*. The study separates two estimands: the *policy-selected documented configuration* performance of each layer (RQ1–RQ3), and a *common-SQL raw-path sensitivity* contrast reporting the residual under identical SQL — changing query formulation, protocol, preparation, round-trips, and mapping together rather than isolating any one. The full harness — adapters, seed data, an autocannon-driven [14] load runner, and a specification-derived expected-result oracle, differential-equivalence checks, and write-state validation — is released as an open replication package.[1]

[1] Harness, deterministic seed, all adapters, raw per-cell measurements, and the table/figure-generating scripts: `https://github.com/mmiotk/express-db-access-performance`; the archived baseline is release v1.12.13 under the Zenodo concept DOI, `https://doi.org/10.5281/zenodo.21313858`; this revision candidate adds the new validation artifacts and corrected-state campaign and will receive a new immutable version DOI before submission.

The case study answers three research questions, each scoped to the benchmarked implementations under the specified operating point:

RQ1 *(performance difference):* How large is the throughput and response-time-p99 *difference* of the benchmarked implementations relative to the native-driver baseline, and how does it vary across the five access patterns? We keep distinct three quantities a fixed operating point conflates: *capacity* (saturating throughput), *high-load latency* (p99 under equal demand), and *latency at matched utilization* (equal fractions of each layer's own capacity).

RQ2 *(backend-stack transfer):* Does the observed layer ordering transfer between the tested PostgreSQL and MySQL backend stacks, where changing the DBMS can also change its supported adapter and lower-level driver?

RQ3 *(pattern spread):* Which of the five tested patterns shows the largest native-relative spread in this configuration?

This paper's primary contribution is a **comparability protocol for access-layer benchmarking**, specified normatively in Section 3.1 (inputs, stages, cell-admission criteria, interpretation, and limits): a concrete discipline for establishing and reporting comparability.

- **Semantic admission.** A seed-specification oracle checks expected read results without using native-driver responses; fixed and randomized differential checks then establish mutual equivalence, and every timed insert must produce the intended database state — field values, generated identifiers, and exact row-count changes, not merely a 2xx. Where the secondary transactional method is implemented, commit atomicity and rollback are checked as well. These checks precede admission of the corresponding timing; a layer that returns less, writes wrong, or errors faster than it works thus cannot appear faster (Section 3).
- **Common-SQL raw-path sensitivity.** The treatment is fixed by a predeclared, reproducible selection rule (*policy-selected documented* here; the proto-

col privileges none), and a *common-SQL raw-path sensitivity contrast* reports how much difference remains once SQL is held fixed — a diagnostic contrast, not a causal attribution.

- **Operating-point separation.** Capacity, high-load tail under equal demand, and tail at matched utilization are reported as *distinct* quantities, not conflated at one load point.

These constituent controls are established performance-evaluation and testing ideas [15, 16, 17]; CYNTHIA already applies differential equivalence across ORM implementations [18]. The claim is their access-layer-specific operational discipline, evaluated by the case-study ablation and a single-rater audit of nine external benchmarks, not invention of the principles or independent inter-rater validation. In this study the specification/state layer exposed shared MySQL state contamination that mutual equality alone would miss; common SQL and matched utilization changed two further interpretations. The machine-readable checklist, source-located audit, and result-blind review packets make that discipline available for independent reassessment.

2. Related Work

We survey prior work along four *coverage* axes — taxonomy, engine, joint throughput/tail reporting, and vendor involvement — spanned by the case study (Table 1). We separately audit which comparability controls each source reports. A structured scoping search (not a systematic review) of the ACM Digital Library, IEEE Xplore, Scopus, and Google Scholar (2006–July 2026) retained empirical comparisons of relational access layers or engines, with search details archived in the replication package (`notes/related-work-search.md`); being scoping rather than exhaustive, we scope every gap claim to the sources searched.

2.1. Object-relational mapping and the access-layer spectrum

Access layers bridge the object-relational impedance mismatch between the relational model and an application's object graphs [19], spanning native drivers, query builders, and ORMs following Data Mapper / Active Record patterns [2] (Section 1). Torres et al. [20] survey two decades of ORM patterns; their trade-off — productivity against a hard-to-predict runtime cost — motivates what we measure while holding the store family (relational rather than non-relational [21]) fixed. The treatment is the configured access-layer implementation bundle, including its selected API, SQL, protocol, and mapping path; the study does not isolate a library binary from those choices.

2.2. ORM performance and data-access anti-patterns

A parallel line studies ORM performance in reverse, repairing the inefficient SQL frameworks emit. The costliest is the N+1 query problem (Section 1); it and related redundant-query anti-patterns pervade real-world web applications [22, 23, 24], their impact quantified in ORM-backed systems [25, 1] and often *removable* by query synthesis, batching, or caching [26, 27, 28] (with a JavaScript `async`/`await` analogue [29]). Closest to our design, Lorenz et al. [30] show the best mapping strategy depends on database technology, van Zyl et al. [31] that it is tuning-sensitive, and a recent study adds an energy dimension [32]. Overhead is thus configurable, not fixed — as our per-pattern results confirm — but this line never compares drivers, query builders, and ORMs head-to-head under load, our aim.

Cross-implementation ORM equivalence checking itself has a direct precedent. CYNTHIA generates an abstract query, translates it to equivalent queries for five ORMs (including Sequelize), executes them over four relational DBMSs, and differentially compares the results to find correctness bugs [18]. It therefore establishes a direct precedent for cross-implementation differential equivalence; matching results alone do not establish correctness against an independent task specification. CYNTHIA is a testing method, however, not a performance-benchmark admission protocol: it does not use equivalence to decide which

timing cells may enter a benchmark, define a premeasurement rule for which implementation represents each library, or separate equal-demand capacity effects from matched-utilization latency. Our claim is restricted to using specification evidence and cross-implementation equivalence as benchmark admission, joined with predeclared treatment selection and operating-point separation.

2.3. Peer-reviewed access-layer comparisons

Peer-reviewed head-to-head access-layer measurement is scarce. Colley et al. [6] demonstrate selected Entity Framework-generated and hand-written SQL in a Contoso application; it is an ORM/non-ORM illustration, not the eight-ORM cross-language benchmark attributed to it in an earlier draft. Three recent studies partially address this: one in *Procedia Computer Science* compares Prisma against optimized raw SQL over eight query patterns on PostgreSQL [7] (differing versions, hardware, and workload, so we neither build on nor dispute its figures); a second compares Prisma, TypeORM, and Sequelize across four relation types, also on PostgreSQL, by response time, memory, and CPU [8]. Pratama and Raharja [9] cross Node-Bench results for five access products with eleven Node.js frameworks and three virtualization/container environments. That study directly varies access products, but framework, access path, and environment also change across cells, so it does not isolate an access-layer estimand. The most recent study times internal query stages of three ORMs rather than client-observed requests [10] and reports its own concurrency-dependent ordering. A further Express study optimizes one stack rather than comparing access layers [13].

2.4. Database-engine, SQL/NoSQL, and Node.js performance

A second, disjoint line benchmarks the layers below and above the access layer without varying it. Salunke and Ouda [11] compare PostgreSQL and MySQL under standard OLTP workloads (the only dual-engine precedent found), but benchmark the *engines themselves*, so cannot say how an access layer changes it; SQL/NoSQL comparisons [33] likewise measure the store, not

the layer. On the Node.js side, work establishes Node's web viability [3], tracks back-end drift under sustained load [4], and compares frameworks broadly [5]. The Node-Bench comparison above also shows why a product count alone is insufficient: its framework–library–environment cells do not hold a common access-layer task fixed.

2.5. Standard benchmarks and benchmarking methodology

Standard benchmarks define how systems are measured: YCSB [34] for cloud-serving stores and TPC-C/TPC-E for OLTP. These target the *engine*, not the driver, query builder, or ORM this study varies, but set the bar for controlled, repeatable load. A methodological literature explains why single-metric reporting fails: Fruth et al. [16] and Raasveldt et al. [15] catalog benchmarking pitfalls (coordinated omission, warm-up); Dean and Barroso [35] show the tail, not the mean, drives user-perceived latency under fan-out; and Li et al. [36] trace its sources. Such measurement is often ad hoc [37, 38] and hard to reproduce [39], motivating a harness built for relevance, repeatability, and fairness [40, 41]. We take a choice absent above: report throughput *and* p99 for the same run, not a single after-the-fact metric.

2.6. Practitioner and vendor benchmarks

A larger body comes from practitioners and vendors, each authored by a party with a stake in a compared product. Drizzle publishes a Northwind micro-suite (nine targets) and a k6 load test reporting req/s, average latency, P95, and CPU [12]. Prisma's site compares Prisma, Drizzle, and TypeORM by median query latency over 500 iterations, reporting neither throughput nor any percentile [42]. IMDBench adds Sequelize and node-postgres, reporting throughput and latency on three CRUD operations [43]. Together these span more products than the peer-reviewed literature, but none combines dual-backend-stack coverage, response-time p99, capacity characterization, and frozen source-located treatment provenance.

2.7. Positioning

In the sources searched, no access-layer benchmark reports all seven protocol dimensions; the source-located audit in Supplement Table S37 records partial controls rather than treating non-reporting as proof of absence, and we found no study combining broad product coverage, both engines, joint throughput/tail reporting, and non-vendor authorship — each covers at most two or three (Table 1), the strongest claim the scoping search supports, not one of priority. But coverage is not the contribution: the protocol operationalizes established experimental principles for this genre, not a new one. Kounev et al.'s systems-benchmarking criteria [44], Hasselbring's software-engineering standard [45], and Raasveldt and Mühleisen's fair-benchmarking guidance [15] supply its principles at a deliberately general level. They leave an access-layer study to operationalize *what code represents a library*, how competing implementations are checked for semantic equivalence, and how strategy-sensitive results are contrasted. Our protocol supplies one concrete operationalization: an arbitrary but predeclared treatment policy; a common-SQL raw-path sensitivity contrast; and a per-cell gate requiring specification-conformant, mutually equivalent responses and verified write state *before* timing. Correctness-before-timing has precedents (SPEC validation, TPC ACID, and SQLancer [46]); cross-implementation equivalence has the direct precedent discussed above. Our narrower contribution uses that check as an explicit prerequisite for admitting benchmark timings and combines it with predeclared treatment selection and operating-point separation. Together these constructs provide a concrete discipline for establishing and reporting comparability among competing implementations of one task. Its evidence is the case-study ablation and a single-rater retrospective audit of nine external benchmarks (Supplement Table S37); this demonstrates utility and applicability, not independent inter-rater validation or invention of the underlying principles.

Table 1: Prior work on relational-database access-layer performance, positioned along four *coverage* axes our case study spans; the paper's contribution is the comparability protocol of Section 1, not this coverage. "Products covered" aggregates native drivers, query builders, and ORMs into one count; "none" marks studies that vary the engine or framework but not the access layer. *Vendor involvement* marks maintainer-authored studies — a conflict-of-interest property of the source, not a methodological guarantee that a non-vendor-authored benchmark is unbiased.

Study	Products covered	Engines	Metrics	Vendor involvement
Colley et al. [6]	one ORM vs. hand-written SQL example	SQL Server	query latency	None
Procedia CS [7]†	Prisma vs. raw SQL	PostgreSQL	latency, CPU	None
JCCT [8]	3 ORMs	PostgreSQL	latency, memory, CPU	None
JCSI [10]	3 ORMs	PostgreSQL	per-stage latency	None
Node-Bench [9]	5 access products (+11 frameworks)	MySQL	throughput, latency	None
Salunke and Ouda [11]	none (engine only)	PostgreSQL + MySQL	throughput, latency	None
Drizzle [12]	9 products	PostgreSQL	throughput, latency, P95	Vendor
Prisma [42]	3 ORMs	PostgreSQL	median latency only	Vendor
IMDBench [43]	4 ORMs + driver	PostgreSQL	throughput, latency	Vendor
This study	**9 policy-selected documented (+2 tuned)**	**PostgreSQL + MySQL**	**throughput + response-time p99**	**None**

†Cited for its methodology only; its absolute speedup figures are not comparable to ours given the differing versions, hardware, workload, and harness.

3. Study Design

In Goal–Question–Metric terms, the study analyzes configured relational-access-layer implementations for the purpose of comparing throughput and response-time p99 under PostgreSQL and MySQL back ends [47]. The viewpoint is the benchmark analyst evaluating policy-defined treatments; it is not a model of observed developer behavior. This is a controlled case study of one synthetic service, dataset, and environment, not inference about ORMs as a population. Following controlled software-engineering guidance [48, 49], the study instantiates the protocol with three decisions: semantic admission before timing, a predeclared treatment-selection policy, and explicit operating points.

The primary RQ1–RQ3 treatment is the *policy-selected documented configuration*: the path selected mechanically from frozen official documentation under the rule below. The protocol itself does not privilege documentation; another study may preregister an expert-tuned, usage-frequency, or vendor-recommended policy. A *common-SQL raw-path sensitivity contrast* then has every portable layer execute the same SQL through its raw facility. SQL, API

level, protocol, preparation, round trips, and mapping can still change together, so this contrast is compound evidence of sensitivity to standardization, not causal decomposition or a bound on library-intrinsic overhead (Supplement Table S42).

A fixed concurrency imposes equal *demand* on layers of unequal throughput and so drives them to unequal *utilization*, so RQ1 is characterized under three operating conditions rather than one privileged point — the protocol's *operating-point separation*. *Equal external demand* is the fixed 50-connection *high-load* point (the full five-pattern, two-engine matrix); a per-pattern concurrency sweep (ladder 1–200, both engines; Supplement Table S35) puts each pattern's throughput knee at or below 50 connections, so this point places every pattern at high utilization of its own capacity (median 97–100%) — a capacity-and-queueing regime rather than a sub-saturation latency comparison — while 50 connections against a ten-connection pool keep about forty requests queued (Controls, below). Capacity is thus *measured* for all five patterns (deep-fetch curve, Supplement Figure S2). The two utilization-dependent conditions need a per-layer capacity target and are shown on the deep fetch: *equal utilization* is the coordinated-omission-corrected open-loop sweep offering each layer a fixed fraction (50/70/85/95%) of its *own* estimated capacity, and an *equal compute budget* is the exploratory equal-CPU check — different questions that need not agree, so all three are reported. For the deep fetch, the experiment defines the capacity proxy $\widehat{C}_{50}$ as the median throughput of the 25-run, 50-connection primary cell; the full sweep subsequently places 50 connections at or above every layer's measured knee. Capacity is estimated rather than known, so Supplement Table S45 evaluates the target against both the within-campaign bootstrap interval and the separately measured sweep maximum. The 50% and most 70% cells remain below saturation; 85% and 95% are interpreted only as near-saturation trends (Supplement Tables S21–S22).

3.1. The comparability protocol

We state the protocol independently of the case study, then instantiate it. It provides a concrete discipline for comparing *treatments* on a fixed *workload*; Figure 1 states its inputs, mandatory and recommended stages, cell-admission rule, and output interpretation. The stages cover the admission-through-interpretation dimensions addressed here; they are not asserted to be a complete taxonomy of every threat to access-layer benchmarking.

Admission versus diagnosis. Semantic admission asks two distinct questions. First, does an implementation satisfy expected results derived from the declared task and deterministic seed rather than from the timed native path? Second, are competing implementations mutually equivalent under the declared comparator? Failure of either read check, or a mutation reaching the wrong database state, excludes the cell. These are finite conformance tests, not proof of correctness for arbitrary inputs. Workload constraints disqualify only when preregistered; query count, joins, and loading strategy are otherwise measured properties of the treatment.

Applicability limits. the output comparator must match the output semantics: byte-identical comparison, used here, is valid only when equivalent outputs are deterministic and canonically serializable, and otherwise (unordered collections, floats, timestamps, nondeterministic identifiers) needs a semantic comparator (set equality, tolerance, canonicalization). The case study's outputs are deterministic and canonically constructed, so byte-equality is valid here.

Compliance levels. M1 semantic admission and M2 treatment definition form the mandatory core because, without a declared task result and a stable configured treatment, there is no common comparison estimand. The remaining stages do not decide whether a fixed-load result exists; they license additional interpretations. Capacity characterization locates that result relative to a throughput knee. The common-SQL raw-path extension licenses only a compound sensitivity contrast; utilization control licenses comparison at stated fractions of esti-

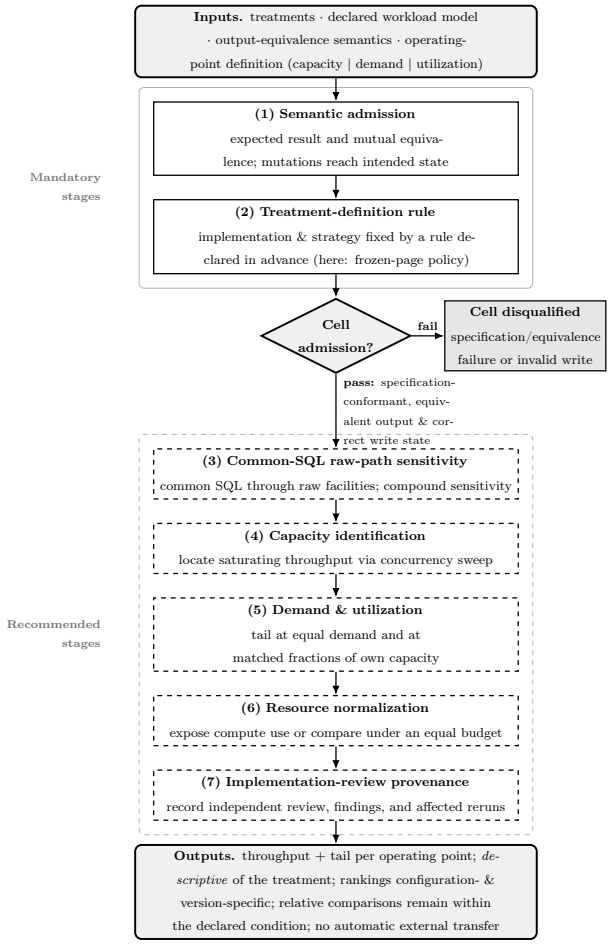

Figure 1: The comparability protocol. Semantic admission and treatment definition form the mandatory core. Recommended stages license common-SQL, capacity, utilization, resource, and implementation-provenance interpretations. A finite gate supplies task-conformance evidence; it does not prove arbitrary-input correctness.

mated capacity; resource accounting licenses a compute-and-memory reading or a labelled equal-budget sensitivity. Implementation-review provenance records evidence against correct-but-needlessly-slow adapter code. In this case study M1–M2 cover the full matrix; R4 covers every pattern; R3, R5, and R6 are scoped to the deep fetch; and R7 remains unsatisfied because no independent human adapter review is claimed. Omission of a recommended stage therefore narrows the supported claim rather than automatically invalidating every reported number.

Table 2 records how each control changed this case study; that establishes consequence here, not universal necessity. Supplement Table S37 separately applies the codebook to nine external benchmark sources. The machine-readable audit preserves 63 source-located judgements, but one author performed the coding, so it demonstrates bounded protocol application rather than inter-rater reproducibility. Blind forms for two treatment-selection raters and an adapter reviewer are shipped under `notes/reviewer-packets`; their status is reported as pending, not silently counted as validation.

3.2. Factors and treatments

We vary three factors. The *access layer* has eleven levels across three tiers (schematic below): two native drivers with two engine-specific tuned baselines (`pg-tuned` reuses named prepared statements, `mysql2-tuned` the binary protocol via `execute()`), the query builder Knex, and six ORMs; the tuned baselines are reference points, not selected treatments, leaving *nine* policy-selected layers.

Tier	Implementations	Runs on
Native driver	`pg`, `mysql2`	its own engine
Tuned native baseline	`pg-tuned`, `mysql2-tuned`	its own engine
Query builder	Knex	both engines
ORM	Drizzle, Prisma, Sequelize, TypeORM, Objection, MikroORM	both engines

The 7 portable layers run on both engines and the 4 native/tuned on one, so $4+7\times2 = 18$ layer×engine cells $\times$ 5 patterns $= 90$ measured configurations. We

Table 2: The comparability protocol, stage by stage. Each row pairs a generic benchmarking principle with its access-layer manifestation, the risk if omitted, and how the stage affected interpretation here. The stages operationalize established principles; the single-rater external audit in Supplement Table S37 evaluates reported coverage without claiming priority. Throughputs abbreviated PG (PostgreSQL) and MySQL; "conns" denotes client connections.

Stage	Benchmark decision	Generic principle	Access-layer manifestation	Risk if omitted	Effect on interpretation here
Semantic admission	Which treatments are timed at all	Expected results and mutual equivalence precede speed	Access-layer paths can return different fields, scalar types, graph shapes, or mutation states; a fast cell may be a silent error doing less work	Wrong or incomplete implementations enter the ranking	The seed oracle supplied 73,080 expected-result checks; the admission layers caught multiple semantic and state defects, including shared MySQL contamination and unequal fan-out seed values
Treatment-definition rule	Which API and loading strategy of each layer is measured	Pre-register and fix the system-under-test configuration reproducibly	Access layers expose multiple official query paths with different round-trip counts; native and builder facilities are fixed explicitly too	Undefined or outcome-informed API choice changes the estimand	Policy-selected documented deep fetch spreads 7.01× (PG) / 5.12× (MySQL); round-trips differ 1–4 by design; Drizzle is the documented tie-break case
Common-SQL raw-path sensitivity	Add a common-SQL raw-path contrast (a compound diagnostic, not a mechanism decomposition)	Standardize query text and plan while disclosing the jointly changed factors	Access-layer time mixes loading strategy with execution machinery; a common raw-path comparison shows sensitivity to standardization but does not separate those mechanisms	Mis-read the whole policy-selected documented spread as library-only "overhead"	The 7.01×/5.12× spread narrows to 1.71× (PG) / 2.03× (MySQL) among raw paths on common SQL under this compound raw-path sensitivity contrast; which component is responsible is not identified
Capacity identification	Locate each layer's saturating throughput before fixing an operating point	Interpret latency relative to capacity, not at an arbitrary fixed load	Configured implementations saturate at different throughputs, so fixed demand places them at different capacity fractions and queue depths	A fixed load compares layers at unequal utilization	A per-pattern sweep (1–200) puts every knee at or below 50 conns; median utilization 97–100% at the 50-conn point; native ∼7× the slowest ORM's capacity
Demand & utilization	Report tail at equal demand and at matched own-capacity fractions, as distinct quantities	Distinguish demand from utilization; correct coordinated omission	Because layer capacities differ, an equal-demand tail conflates queueing with sub-saturation latency; engines at similar capacity make the two nearly coincide	Read a capacity/queueing gap as a sub-saturation latency difference	At 50 conns deep-fetch p99 runs pg 18 ms to MikroORM 108 ms; at nominal 50% of the $\widehat{C}_{50}$ capacity proxy the p99 band contracts to 2.3–4.9 ms, and the full-sweep-denominator sensitivity stays below saturation (S45)
Resource accounting	Report performance with compute/memory demand or under an equal budget	Expose resource demand alongside speed	Similar throughput can consume different application or database CPU	A speed ranking hides its compute cost	CPU, RSS, requests per CPU-second, and an exploratory equal-CPU deep-fetch run qualify the throughput result
Adapter review	Record independent review and premeasurement findings	Separate implementation quality from semantic conformance	Equivalent code can still allocate, convert, or serialize needlessly	Correct-but-inefficient adapter code is attributed to the library	Not satisfied here: all adapters and assignments are single-author; blind review packets are shipped and this remains a validity threat

Table 3: Estimands defined in this study, consolidating the identification qualifications stated across Sections 3 and 4. Each quantity is *descriptive* of the treatment as defined; only relative within-condition differences are meant to travel, and rankings are configuration- and version-specific.

Estimand / quantity	**What it identifies**	**What it does *not* identify**	**Where reported**
Policy-selected documented configuration performance (RQ1, primary)	Performance of the configuration selected by the frozen-page policy (throughput, p99)	Library-only overhead; the most common or most performant usage	Tables 4, 5
Common-SQL raw-path sensitivity contrast (diagnostic)	Residual spread once every layer runs common SQL through its raw facility	A causal decomposition or a bound on the library effect; the share from SQL formulation or query count	Supplement Table S42
Performance-conscious deep-fetch regime	Speed recoverable within a narrow byte-equivalent tuning budget (faster documented loading strategy)	Gains from SQL rewrites, caching, or schema change; non-drop-in strategies (excluded)	Supplement Table S18
Per-pattern capacity (saturating throughput)	Each pattern's throughput knee per layer and engine, locating relative utilization at the operating point	Tail latency; a throughput quantity, not a demand- or utilization-dependent comparison on its own	Supplement Table S35, Figure S2
High-load p99 under equal closed-loop demand (primary tail)	The tail a deployment sees at fixed 50-connection demand, including acquisition queueing	The tail at comparable utilization; the pooled cross-run p99	Table 4
Matched-utilization tail (p99 at matched fractions of own capacity)	p99 when each layer is offered 50% and most 70% of its estimated capacity; 85–95% cells are secondary near-saturation trends	The tail under equal external demand; a precise claim near 85–95% utilization	Supplement Tables S43, S45
Layer×engine interaction magnitude (RQ2)	The spread of each layer's PostgreSQL÷MySQL stack ratio and the concrete rank reversals	A pure DBMS effect; stability across campaigns, hosts, or deployments	Supplement Table S28; §4 (RQ2)

classify a library by its dominant interface, not its self-description (Section 1). The nine policy-selected layers are a deliberately broad product cross-section, not balanced sampling of the three tiers and not an exhaustive catalogue — each maintained, used, and (portable tiers) running on both engines — excluding non-portable PostgreSQL-only Slonik and pg-promise and query-builder Kysely (covered by Knex; `METHODOLOGY.md` gives the criteria). *Non-vendor authorship* means only that no evaluated library's maintainers were involved; the consequential author choices (selection, schema, pool size, tuning) remain and are disclosed. All adapter code and all treatment assignments were produced by the single author. Semantic admission cannot detect a semantically correct but inefficient implementation. The artifact therefore records this as an internal-validity limitation, adds implementation-review provenance as a recommended protocol stage, and ships result-blind review packets; no maintainer or independent human sign-off is claimed. The *backend-stack* factor has two nominal DBMS levels (PostgreSQL 18.4 and MySQL 9.7.1), but switching it can also switch the supported adapter, lower-level driver, wire protocol, and defaults; RQ2 therefore estimates transfer between these stacks rather than a pure DBMS effect. The *access pattern* has five levels, realized as HTTP endpoints (Supplement Table S39). Supplement Table S29 records which factors are held constant, vary, or are uncontrolled on this single virtualized host — the reason we report this configuration rather than general library performance.

3.3. Objects: schema, workload, and data

The workload is a minimal blog domain (`authors` $1{-}*$ `posts` $1{-}*$ `comments` $*{-}1$ `authors`). The five endpoints implement five representative access patterns [34] (Supplement Table S39): a keyset page (primary-key cursor, not a large `OFFSET`), a single-row read, a one-to-many list, the deep fetch (a post with its author and every comment, each with its own comment-author), and a per-author aggregation. A deterministic seed (2,000 authors, 100,000 posts, 1,000,000 comments) is loaded by the native driver, independently of any layer under test, so both engines receive byte-identical row values; on-disk representa-

tion, index layout, and collation naturally differ. The working set fits in memory, so measurements emphasize the access layer's per-request instruction, protocol, and mapping cost, disk-read latency largely removed, while engine-side execution, buffering, locking, and logging remain in every measurement [50]. Each request's target identifier is drawn uniformly from the seeded range (posts 1–100,000, authors 1–2,000), spreading load across the working set; a skewed, production-like distribution is a planned variant (Section 6).

3.4. The adapter contract and N+1 control

Every access layer implements the same factory interface (five query methods plus a teardown), so the server and load runner are layer-agnostic and each returns an *identical result shape*. Each avoids the deep-fetch N+1 problem by its selected mechanism — a hand-written join for the native drivers, Knex, and query-builder-style Drizzle (tuned baselines reuse it), and relation eager loading (`include`, `populate`, `relations`, ...) for the data-mapper ORMs. The comparison is thus of each layer's *selected configuration* under a fixed adapter-selection rule, not a fixed query plan: a measured difference reflects the SQL generated, the round-trip count, and result materialization. Server-side logging captures the exact SQL, round-trip count, and `EXPLAIN` plans; per-endpoint counts differ by design (Supplement Table S2).

The API was fixed by a rule decided *before* measurement: the eager-loading API each layer's version-compatible official documentation presents first, ties broken (as for Drizzle's two builders) toward the lower-level API that the library documents as its base layer. This is an intentionally artificial, predeclared *policy*, not evidence that the chosen API is performance-optimal or the most common production choice — one instance of the protocol's predeclared treatment rule (Section 3.1), which privileges none; a study targeting expert-tuned usage would substitute its own (corroboration and limits: Table S33, Section 6). Supplement Table S16, `METHODOLOGY.md`, and the rubric (`notes/documentation-selection.md`) record, per layer, that API (documentation page, version, URL), its round-trip count, the documented alternative not

used, that logging, hooks, and validation are off, and the freeze date. The artifact ships the exact HTML response returned by the nearest available pre-freeze Wayback capture of each official URL, together with its capture timestamp and SHA-256 (`experiments/documentation-snapshots/manifest.json`), so reconstruction does not depend on today's website. A capture proves the page state at its recorded timestamp, not that it remained unchanged through the 15 July 2026 freeze; the selection record therefore distinguishes the freeze date from the evidence date. If official pages conflict, the version-matched relations/eager-loading page controls over a quick-start or marketing page; remaining equal prominence invokes the declared documented-base-layer tie-break, and an unresolved tie must be reported as ambiguous.

For the protocol's *common-SQL raw-path sensitivity contrast*, every adapter also implements: the same two-statement deep-fetch SQL and row mapping (per engine, the same `EXPLAIN` plan) through the layer's raw-SQL facility, mirroring the aggregation control. Server-side capture, synchronized to a request marker so handshakes are excluded, confirms these statements are byte-identical across every layer on both engines; only the wire protocol differs (Supplement Table S14). A secondary loading-strategy sensitivity switches only between documented, byte-identical deep-fetch alternatives; it does not redefine the primary policy-selected treatment or estimate a library-intrinsic effect (Supplement Table S18).

Read admission has an expected-result layer independent of any timed reference. `bench/verify-spec.mjs` replays the documented seed PRNG and campaign fan-out fixture in memory, without querying a database or importing an access layer, and derives exact fields, values, ordering, graph membership, and aggregates. Database-generated timestamps are checked for presence and canonical ISO-8601 representation rather than a predicted instant. Across 1,000 random inputs per identifier type, boundary and list cases, and all six fan-out fixtures, all nine adapters on each engine pass 73,080 expected-result checks (Supplement Table S38).

A separate differential layer asks whether implementations agree. Shared

canonical constructors fix fields, key order, and scalar types; they copy already materialized rows and do no joining, regrouping, sorting, or caching. A 30-block standalone measurement places the ten-comment deep-fetch constructor at 9.35–9.36 μs, or 0.072% of the smallest corresponding HTTP p50. The fixed probes in `verify.mjs` and randomized edge/input sweep in `verify-property.mjs` then compare serialized outputs across implementations, with no divergence (Supplement Table S38). The specification and differential layers provide finite evidence for the declared task and tested inputs; neither is presented as proof for arbitrary inputs. A companion check (`bench/verify-writes.mjs`) validates database *state*, not HTTP status: a separate out-of-band native-driver connection confirms, for every adapter on both engines, that the primary single-row autocommit insert wrote exactly the requested values and identifier. For the adapters that implement the exploratory transactional method (five on PostgreSQL, four on MySQL), it additionally confirms that the post and five comments commit atomically and that an invalid final comment rolls back to no partial state; all declared methods pass. The per-adapter scope is preserved in `write-admission.json`. The read cross-check caught two defects during development (a fan-out aggregation bug and a driver result-shape mismatch; Section 5).

3.5. Controls

Four controls hold the shared conditions fixed — connection pool, logical task, physical dataset state, and observer — so the *configured access-layer implementation bundle* is the only *deliberately varied* factor; they do not isolate the library alone: the bundled differences in SQL, query count, protocol, loading strategy, and hydration are the treatment (Supplement Table S42), not confounds to remove. *(i) Pool size* is fixed at ten connections for every adapter, so the comparison reflects the access-layer choice, not pool tuning [51, 52] (Prisma's core-count-derived default is pinned via `connection_limit`). Because the load generator opens 50 concurrent connections against this ten-connection pool, roughly forty in-flight requests always wait in each library's acquisition

queue; this queueing is deliberately measured and held identical across layers (a 200 ms sampler confirmed the native PostgreSQL pool fully occupied, about 32 requests pending). *(ii) Common task semantics*: each endpoint implements the same declared input–output task, while SQL text, query count, plan, protocol, loading strategy, and hydration remain measured properties of the configured treatment. Aggregation is the exception with a common correlated-subquery formulation; the separate common-SQL raw-path contrast standardizes SQL only for the deep fetch and is interpreted as compound. *(iii) Write isolation*: the declared reset rebuilds PostgreSQL from its seed template and uses a disclosed in-place delete/`OPTIMIZE TABLE`/allocator reset on MySQL; generated rows are removed before warm-up, measurement, and the next cell. A post-review state oracle found that the historical MySQL rebuild ran `OPTIMIZE TABLE` and then restored `AUTO_INCREMENT` to `MAX(id)+1`; because the fan-out fixture ended at 250006, generated rows fell below the 300000 cleanup floor and could persist. The corrected harness inserts and deletes a 300000 sentinel after seeding/rebuild, and every logical reset—including between warm-up and measurement—deletes generated rows and restores the next identifier to exactly 300001. An idempotent preflight verifies exact base/fan-out counts and the allocator before and after a campaign. Historical MySQL range-scan outputs near the base-key boundary, aggregation results, and insert state may therefore include undeclared rows and are not used as clean evidence. A further cross-engine gate compares every declared fan-out value and generated identifier after excluding only DBMS-generated timestamps. It exposed a second shared-fixture defect: an advancing PRNG had produced different comment-author assignments across engines despite identical comment counts. The seeder now materializes the fixture once and the 662-row joined representation is byte-identical across engines (SHA-256 `402e0aebbbc99f6ca32f6e176182fbb4fe753fe273f2e79fb00d5b73a99fe5ca`). The affected pilot was rejected. A corrected-state replacement campaign remeasures all five patterns for all 18 compatible treatment–stack pairs under one version of the instrument. *(iv) Observer separation*: the HTTP load

generator runs in a process separate from the server.

3.6. Measurement procedure

Load was generated with `autocannon` [14], a closed-loop (constant-concurrency) HTTP/1.1 generator [53] recording a latency histogram. The accepted corrected-state primary matrix contains 90 configurations with 25 repeated runs each; the superseded campaign remains provenance only. With `INDEP=1`, each replicate randomizes adapter–engine blocks and boots one fresh server process for the selected read endpoints, which run sequentially in that process; the write endpoint uses another fresh process after the declared state rebuild (a PostgreSQL template restore or the disclosed MySQL in-place rebuild). Thus no application process crosses an adapter, engine, or replicate boundary, but read endpoints within one block share that process, and database buffers plus host/hypervisor state are not reset. At teardown, the server stops accepting requests, awaits every tracked handler promise, closes the adapter pool, and the runner awaits process exit. Each endpoint receives a discarded 15-second warm-up followed by one 12-second measurement at 50 connections. A forward/reverse-order write-cell check found no detectable order effect. The campaign records its ID, deterministic order seed, environment fingerprint, and exact source manifest.

Within a replicate, every layer and engine receives the same request-ID sequence from a PRNG seeded on the endpoint and replicate. The inferential unit is the endpoint-level run; individual requests are subsamples. We report the median of the 25 replicate values of throughput and of the run-level percentiles p50, p90, p97.5, and p99. Throughout, *p99* denotes this *median run-level p99* — the median across replicates of each run's own p99 — not the p99 of the pooled request distribution, which the design does not estimate. Ten 60 s re-measurements of the deep fetch give 12-versus-60-second rank correlations of 1.00 on both PostgreSQL and MySQL; the largest cell-median shifts are 5.6% and 3.8%, respectively (Supplement Table S23). This is endpoint- and campaign-specific sensitivity evidence for the 12 s result, not a general claim

that 12 s is sufficient for p99 estimation. We sampled server CPU and peak RSS over the process tree, with database and load-generator CPU separately. Inserts were measured under each engine's *default* durability (PostgreSQL `fsync` and `synchronous_commit` on; MySQL `innodb_flush_log_at_trx_commit=1`, `sync_binlog=1`), the regime a deployment runs unless relaxed; a labelled secondary run relaxed all of these to isolate the durability mechanism (Supplement Table S3). To characterize behavior under load we swept the deep fetch across 1–256 connections for every layer and engine [54]; and a native `undici`-based constant-arrival generator drove the deep fetch under a fixed offered rate (250–4,000 req/s) rather than fixed concurrency, measuring from intended send time and clipping timeouts at their cutoff — a coordinated-omission-corrected design [55, 56] that checks the closed-loop tail's sensitivity to a different load model (Section 4).

3.7. Analysis

We separate *primary* outcomes, which carry the research-question claims, from *secondary* and *exploratory* outcomes reported descriptively as controls and sensitivity analyses (Supplement Table S34): the primary outcomes are throughput and the closed-loop p99 on the five patterns against both engines at the fixed operating point.

Because absolute throughput is hardware-specific, we analyze *relative* differences within an engine; a pattern's *spread* is the untuned native driver's median throughput divided by that of the slowest layer on that pattern (native-relative, so comparable across patterns). We report medians over the 25 runs, the coefficient of variation as a stability signal [17, 57], and a fixed-seed percentile bootstrap 95% CI for the median; these intervals capture within-campaign, run-to-run variability on this host and configuration, not variation across machines, versions, or deployments (Section 6). The design is a *randomized block* — within each replicate every layer runs the identical request stream — so adjacently ranked layers are compared with *paired* tests on their per-replicate throughput ratios (a two-sided sign-flip permutation test cross-checked with Wilcoxon

signed-rank), reporting the geometric-mean paired ratio, its bootstrap CI, the paired dominance, and a TOST against a ±5% margin (author-selected and application-dependent, not universal; supplement) for the closest pair; we foreground these effect sizes and interval estimates over p-values [58, 59]. Because the seven adjacent pairs follow the *observed* ranking, their significances are a post-hoc, descriptive robustness check, not a confirmatory family. Adjacent-layer p99 gaps of one or two milliseconds sit at the millisecond measurement floor and are not read as real; conclusions rest on the large-ratio patterns, chiefly the deep fetch. The full construction (procedure and seed, tie handling, the layer×engine interaction test, rank correlations, equivalence margin) is in the statistical-methods notes (`notes/supplement-methods`); paired p99 comparisons are Supplement Table S30.

3.8. Experimental setup

Measurements were taken 2026-07-16 to 2026-07-18 on a single host — a virtualized Intel Xeon Gold 6140 instance (16 vCPUs, single NUMA node, 126 GB RAM; Linux 6.17; Node.js 24.18.0; Express 5), PostgreSQL 18.4 and MySQL 9.7.1 local — the primary matrix overnight and fourteen labelled secondary experiments (order-invariance, durability, same-SQL, open-loop, fan-out, equal-CPU, concurrency, utilization, pool-size, cluster, mixed-workload, wait-event, post-restart, longer-window tail) over the same window. Each library is pinned to the *latest stable release compatible with the harness adapter contract* as of the freeze date (2026-07-15), recorded from the committed lockfile and an automated environment capture (Supplement Table S11); only Sequelize invokes the earlier-release exception (its version-7 line is a prerelease, so the current 6.x stable line is used). Because access-layer performance is version-sensitive, this pins a dated snapshot rather than a permanent ranking (Section 4).

3.9. Replication package

The harness — the Express application, the adapter contract, all eleven adapters, the deterministic seed, the `autocannon` matrix runner, the semantic-admission checks, and the scripts regenerating every table in Section 4 — is

released so the full matrix re-runs with one command against a containerized or local PostgreSQL and MySQL.

4. Results

Tables report throughput (req/s, higher better) and tail latency (p99 ms, lower better) per access layer and engine. The two consequential patterns are in the main text (deep fetch, Table 4; insert, Table 5); the point read, keyset range scan, and aggregation are in Supplement Tables S12, S13, and S15, all from the run of Section 3. Each native driver has an untuned policy-selected row (`pg`, `mysql2`) and tuned (`pg-tuned`, `mysql2-tuned`), the tuned rows marking a hand-tuned reference point, not one attained without code changes. We compare *relative* differences within an engine, where two estimands recur: the *performance of the selected configured treatment* (RQ1–RQ3), rather than library-only overhead; and the *common-SQL raw-path sensitivity contrast* (Supplement Table S14), which reports the residual spread when every layer runs common SQL through its raw facility (construct validity: Section 6).

4.1. RQ1: performance difference relative to the native baseline

The native implementation leads each of the ten engine–pattern comparisons: `pg` or `pg-tuned` is fastest on all five PostgreSQL patterns, and `mysql2` or `mysql2-tuned` on all five MySQL patterns. Among the selected treatments, the corresponding untuned native driver leads each comparison. The current-generation ORMs sit mid-to-low on the read patterns; Prisma is near the bottom of point reads and aggregation on both engines (Table 4; Supplement Tables S12 and S15), retiring an earlier version's headline of Prisma tying the native driver at a large CPU premium (Section 5).

`pg-tuned` and `mysql2-tuned` correspond closely to their untuned counterparts within a few percent except on the deep fetch, where a second round trip lets prepared statements pay off (`pg-tuned` 3,746 vs. `pg` 3,638; `mysql2-tuned` 2,415 vs. `mysql2` 2,416); on PostgreSQL aggregation `pg-tuned` adds about 8% (7,538 vs. 6,978).

Table 4: Deep fetch: throughput (req/s, higher is better; median with a within-campaign 95% bootstrap interval over the 25 repeated runs — run-to-run variability within the source campaign of each cell, not across hardware or days) and response-time p99 (ms, lower is better; median run-level p99 with the same within-campaign 95% bootstrap interval) by access layer and engine. p99 is reported at 1 ms resolution, so cells differing by ≤ 1 ms are practically unresolved (see the paired significance analysis in the supplement).

		PostgreSQL		MySQL	
Layer	Category	req/s [95% CI]	p99 [95% CI]	req/s [95% CI]	p99 [95% CI]
pg	native-driver	3638 [3602–3662]	18 [17–20]	–	–
mysql2	native-driver	–	–	2416 [2413–2425]	28 [27–28]
pg-tuned	native-tuned	3700 [3656–3773]	18 [16–19]	–	–
mysql2-tuned	native-tuned	–	–	2398 [2386–2419]	25 [24–26]
knex	query-builder	2431 [2415–2459]	27 [25–28]	1982 [1963–2006]	30 [29–31]
drizzle	orm	1829 [1815–1846]	34 [33–35]	1301 [1290–1308]	48 [46–49]
prisma	orm	1175 [1160–1179]	52 [51–53]	630 [623–638]	91 [89–94]
sequelize	orm	978 [973–983]	60 [60–61]	880 [871–886]	68 [67–68]
typeorm	orm	1219 [1205–1233]	49 [48–50]	1017 [1008–1030]	57 [56–58]
objection	orm	1017 [1011–1029]	56 [54–58]	836 [824–849]	70 [68–70]
mikroorm	orm	519 [512–528]	108 [107–112]	472 [464–475]	121 [118–126]

Table 5: Insert: throughput (req/s, higher is better; median with a within-campaign 95% bootstrap interval over the 25 repeated runs — run-to-run variability within the source campaign of each cell, not across hardware or days) and response-time p99 (ms, lower is better; median run-level p99 with the same within-campaign 95% bootstrap interval) by access layer and engine. p99 is reported at 1 ms resolution, so cells differing by ≤ 1 ms are practically unresolved (see the paired significance analysis in the supplement).

		PostgreSQL		MySQL	
Layer	Category	req/s [95% CI]	p99 [95% CI]	req/s [95% CI]	p99 [95% CI]
pg	native-driver	6018 [5847–6185]	12 [11–13]	–	–
mysql2	native-driver	–	–	1766 [1451–1844]	116 [105–127]
pg-tuned	native-tuned	6125 [5868–6254]	12 [11–14]	–	–
mysql2-tuned	native-tuned	–	–	1683 [1496–1797]	117 [106–124]
knex	query-builder	4581 [4490–4715]	16 [14–18]	1779 [1418–1806]	118 [105–125]
drizzle	orm	4091 [4010–4163]	15 [15–16]	1816 [1624–1837]	105 [98–114]
prisma	orm	2695 [2675–2740]	24 [23–25]	894 [859–912]	129 [116–144]
sequelize	orm	2616 [2564–2649]	23 [22–24]	1503 [1430–1696]	119 [108–134]
typeorm	orm	2567 [2526–2602]	25 [24–26]	1355 [1248–1388]	115 [108–117]
objection	orm	3780 [3686–3852]	17 [16–20]	1807 [1514–1818]	108 [101–118]
mikroorm	orm	1894 [1848–1915]	32 [31–34]	1237 [1103–1282]	110 [102–114]

On the **point read** the native-relative spread is 2.83× (PostgreSQL) and 2.14× (MySQL) (Supplement Table S12). On the **deep fetch** (a post, its author, and all comments with their authors) it is the widest measured — 7.01× on PostgreSQL (`pg` 3,638, 95% CI [3,602–3,662], down to MikroORM's 519 [512–528]) and 5.12× on MySQL (`mysql2` 2,416 [2,413–2,425] down to MikroORM's 472 [464–475]). The native driver leads on both engines, the data-mapper ORMs (especially MikroORM) trailing furthest (Table 4); the comparison is paired (Section 3), every adjacent MySQL step large relative to its bootstrap interval (Supplement Table S36). **Aggregation** is the least layer-sensitive pattern: the native-relative spread is only 1.82× (PostgreSQL) and 1.87× (MySQL) (Supplement Table S15). Every layer forwards a single query for this pattern, which also avoids the deep fetch's nested-graph materialization; the design does not isolate either feature as the cause of the smaller spread.

4.2. Common-SQL raw-path sensitivity contrast

Server-side capture confirms common statement text and database plans across raw paths on both engines; API level, wire protocol, preparation, round trips, and mapping can still differ (Supplement Table S42). The deep-fetch native-relative spread is then far smaller — 1.71× (PostgreSQL) and 2.03× (MySQL) against the selected-configuration 7.01× and 5.12× (Supplement Table S14). Thus the selected paths differ more than these common-SQL raw paths; the compound contrast assigns no share to any mechanism. A loading-strategy sensitivity check (Supplement Table S18) argues against a mere selected-strategy artifact: forcing the join-selected ORMs (Sequelize, MikroORM) to select-in makes them slower, and Objection's batched-select-in selected path runs about 1.6× slower than a single join on MySQL. Objection's result is therefore sensitive to the documented loading option; this comparison does not isolate a library-only cost.

4.3. RQ2: backend-stack transfer of the ranking

Switching the backend changes a bundle that can include the DBMS, supported adapter, lower-level driver, wire protocol, and defaults. RQ2 therefore

concerns *backend-stack transfer*, not a causal DBMS effect. In the primary campaign, the seven portable layers have stack-ratio heterogeneity on every pattern (blocked interaction $F = 127.1$–458.5, permutation $p = 0.00005$; Supplement Table S28), but this within-campaign test does not establish durable rank reversals.

A second same-host campaign independently randomizes 25 repetitions of the seven portable layers for deep fetch, aggregation, and insert (Supplement Table S46). Deep-fetch ranks reproduce exactly on both stacks (Spearman $\rho = 1.00$, maximum median drift 2.3%); aggregation reproduces 5/7 PostgreSQL and 7/7 MySQL positions ($\rho = 0.96$ and 1.00). MySQL insert is less stable: only 3/7 positions reproduce, $\rho = 0.82$, the leader changes from Drizzle to Knex, and maximum drift reaches 13.6%. We therefore promote a cross-stack reversal only when it occurs in both campaigns, exceeds the predeclared 5% practical margin on both stacks, and paired bootstrap intervals exclude equality in opposite directions in both campaigns. Four pairs meet all criteria: on deep fetch, Prisma exceeds Objection and Sequelize on PostgreSQL but trails both on MySQL; on aggregation, Prisma exceeds Objection on PostgreSQL but trails it on MySQL; on insert, Prisma exceeds MikroORM on PostgreSQL but trails it on MySQL. Other changes in close ranks, including the identity of the MySQL insert leader, remain exploratory.

The magnitude of stack transfer is also treatment-specific. Primary-campaign PostgreSQL÷MySQL ratios range from 1.10 to 1.84× on deep fetch, 1.27 to 1.82× on aggregation, and 1.55 to 3.06× on insert (Supplement Table S28). These ratios and the recurring reversals show that the observed ordering cannot be assumed to transfer unchanged between the two tested stacks. They do not allocate the difference to the DBMS, adapter, driver, protocol, or defaults.

The primary MySQL insert medians illustrate why fine ordering is not a headline: Drizzle, Objection, Knex, and the native driver lie between 1,766 and 1,816 req/s, while several replicate distributions are visibly multimodal or heavy-tailed and reach 13.6% CV (Figure 2). Prisma is the slowest policy-

selected treatment at 894 req/s, giving a 1.98× native-relative spread, but the broad interval on that spread is [1.65–2.09]. Under relaxed durability the native MySQL and Knex paths gain 2.42× and 2.07×, whereas gains differ by treatment; concurrent wait-event sums are consistent with a shared commit-path contribution but are not a causal time fraction (Supplement Tables S3 and S19). PostgreSQL insert spans 3.18× (6,018 to 1,894 req/s) and changes by at most 14% in the relaxed-durability sensitivity. These compound interventions do not decompose backend-stack cost.

4.4. RQ3: which pattern spreads widest

The primary native-relative spreads on PostgreSQL are deep fetch (7.01×), keyset range scan (4.22×), insert (3.18×), point read (2.83×), and aggregation (1.82×). On MySQL they are deep fetch (5.12×), range scan (3.88×), point read (2.14×), insert (1.98×), and aggregation (1.87×). Thus the tested deep fetch has the largest spread and aggregation the smallest on both stacks. A full concurrency sweep places the median knee at 8–32 connections and the median utilization at 50 connections at 98–100% for every pattern–stack combination; the lowest individual values are 87–88% for two PostgreSQL patterns (Supplement Table S35). The fixed point is therefore a high-utilization comparison throughout, although it is not an identical utilization fraction for every layer. An exploratory fan-out sweep from 0 to 500 comments finds a roughly fixed multiplier rather than a spread that grows with breadth; it does not vary graph depth, schema, or query mix.

4.5. Tail latency

At equal closed-loop demand, deep-fetch p99 spans 18–108 ms on PostgreSQL and 28–121 ms on MySQL (Table 4). The run is the analysis unit; a 12 s slow-layer run supplies only about 60 observations to its top percentile, so we use the median of 25 run-level p99 values and show their raw spread. Ten 60-second repetitions preserve the practical ordering, with 12-versus-60-second

rank correlation 1.00 on both engines; the largest median shift is 5.6% on PostgreSQL and 3.8% on MySQL (Supplement Table S23). This is endpoint-specific sensitivity evidence, not a general sufficiency claim for a 12-second p99 window.

At the stable nominal 50% capacity target, coordinated-omission-corrected p99 contracts to 2.3–4.9 ms on PostgreSQL and 1.9–5.5 ms on MySQL; most 70% cells remain similarly small (Supplement Tables S21–S22 and S43). Re-expressing the targets against the full-sweep maximum places nominal 50% at 46.6–51.0% and nominal 70% at 65.2–71.5%; repeated sweep curves widen those ranges only to 46.2–52.2% and 64.6–73.0% (S45). The 85% and 95% results are unstable near-saturation sensitivity observations and do not support a convergence claim. The stable 50% result shows that the large equal-demand p99 spread contains a material capacity-and-queueing component; it does not identify its share or imply a deployment-general sub-saturation bound.

4.6. Measurement stability and effect sizes

Deep-fetch throughput CV reaches 4.5%, much smaller than its multi-fold spreads. The largest primary throughput CV is 13.6% on MySQL insert, whose raw replicates are shown because a median and interval can hide regime mixtures. All 90 cells contain exactly 25 run-level samples and zero timed errors, timeouts, or non-2xx responses. One pre-warm-up startup failure was retried once and recorded; no timed sample preceded that retry. The fail-closed roster validation requires 90×25 observations before accepting the file.

Predeclared native-driver contrasts are foregrounded; ranking-selected adjacent comparisons remain descriptive. Paired bootstrap intervals characterize only within-campaign variation, while the same-host validation campaign is reported separately. Differences of about 1 ms in p99 and throughput ratios inside the predeclared $\pm5\%$ practical margin are not promoted as substantive ordering.

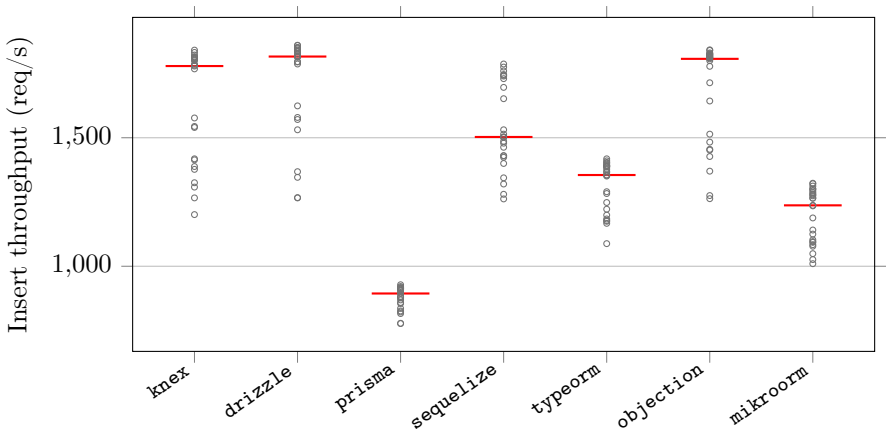

Figure 2: Corrected-state raw per-replicate MySQL insert throughput for the seven portable layers (25 independent repetitions; red bar = median; maximum CV 13.6%). The points, rather than only medians and intervals, expose any mixture or tail structure. Historical insert samples affected by the reset defect are not used for the corrected ordering.

5. Discussion

Version-sensitivity. An earlier campaign found Prisma (then 5.22, on an in-process Rust query engine) tying the native driver on the deep fetch while drawing about five application cores; the *identical* harness on the latest compatible stable versions as of the 15 July 2026 freeze did not reproduce it — application CPU now stays near one core rather than the earlier multi-core path, and Prisma sits at mid-to-low throughput (Section 4). The non-reproduction is a fact, its cause a hypothesis (Prisma's Rust-free rewrite is likeliest, though the re-freeze moved the whole toolchain at once), so the headline is version-sensitive [31, 60].

Implications for practice. RQ1 and RQ3 agree the selected-configuration difference is specific to the access *pattern*: the native-relative spread is widest on the deep fetch — where a layer must hydrate rows and resolve associations rather than merely issue SQL (the object-relational impedance mismatch [19] made concrete) — and narrowest on aggregation. Round-trip count alone does not explain throughput: on the PostgreSQL deep fetch, single-query MikroORM is slowest while two-query Drizzle outruns four-query Prisma (Supplement Ta-

ble S2; Table 4). Result-graph materialization and the other bundled implementation differences are plausible contributors, but the study does not isolate them.

Single-row inserts and backend-stack transfer. Single-row insert behavior is both stack- and treatment-specific. In the primary campaign PostgreSQL spans 3.18×, whereas MySQL spans 1.98× and several MySQL replicate distributions are multimodal or heavy-tailed. Relaxing durability raises the native MySQL and Knex paths by 2.42× and 2.07×, but treatment ratios range from 1.02 to 2.44×; aggregate concurrent wait-event time can exceed wall time and is not a causal fraction. Together these controls are consistent with a shared commit-path contribution, not with attributing a performance floor or a fixed share to the DBMS (Supplement Tables S3 and S19).

The second same-host campaign preserves PostgreSQL insert ranks but only 3/7 MySQL positions; its MySQL leader changes from Drizzle to Knex. We therefore do not claim a durable fine ordering for MySQL insert. The recurring, material insert reversal is narrower: Prisma exceeds MikroORM on PostgreSQL and trails it on MySQL in both campaigns, with paired intervals excluding equality in opposite directions (Supplement Table S46). This supports backend-stack sensitivity in the tested implementations while leaving cross-host transfer unresolved. A five-replicate, representative-layer six-statement transaction remains exploratory and does not extend the single-row-insert finding to writes as a class (Supplement Table S8).

Methodological lessons: a checklist for access-layer benchmarking. Several failures changed or could have changed the comparison; we distil them into a reusable checklist (`notes/supplement-methods`): an **aggregation fan-out** (a join before `SUM` inflating the aggregate) and a driver result-shape mismatch, both semantic defects caught by specification/differential admission; **OFFSET pagination** collapsing on MySQL, easily misread as an access-layer weakness; **write-state contamination**: the post-review state oracle found that MySQL allocated inserts below the cleanup floor after the fan-out fixture; and a **query-**

formulation confound where a mis-scoped pre-aggregation re-scanned the whole table, a query-plan artifact masquerading as an access-layer effect [61]. The write path needs its own gate: MikroORM 7's dropped `persistAndFlush` made every insert throw a 500 faster than a real one, briefly *leading* MySQL until the non-2xx counter flagged it. Because a 2xx alone would miss a *silently* incorrect write, the gate also validates post-write database state — field values, identifiers, row-count changes, and rollback (Section 3) — the write-state check complements the specification-derived and differential read checks. None is schema-specific; each extends the database-benchmarking pitfalls of Fruth et al. [16] and Raasveldt et al. [15] to the access-layer sub-genre, and at least one plausibly explains part of the gap between some vendor-reported and controlled benchmark results.

Evidence for the protocol contribution. The case-study record shows concrete consequences of the protocol stages: an independent seed specification caught shared state/fixture defects that mutual equality would miss; treatment provenance made the loading-policy sensitivity auditable; capacity characterization changed the tail interpretation; and the common-SQL path narrowed, but did not decompose, the selected-treatment spread (Table 2). A codebook was also applied to the nine external benchmark sources retained by the scoping search. Its 63 source-located judgements yield explicit claim licenses rather than a binary quality score (Supplement Table S37). This demonstrates that the protocol can structure an external evidence audit; because the manuscript author was the sole rater, it does not establish inter-rater agreement or independent general applicability. The methodological claim is consequently an access-layer-specific operational discipline and reusable artifact, not invention of the constituent controls or a complete theory of benchmarking validity.

Practical guidance. The synthesis follows from RQ1–RQ3 and describes the benchmarked implementations in this configuration and workload — single host, default durability, pinned versions (Section 6) — not access-layer categories in general. Because the fan-out sweep (Section 4) varies breadth (0–500 children)

but not depth, schema, or query mix, the selected spread's peak on the deep fetch locates a difference among five probes on one synthetic schema — a benchmark observation to re-measure on the target workload, not an established law. Where a hot path resembles the tested deep fetch the access layer is consequential and worth benchmarking: the thin paths (native driver, Knex) led here *for throughput and response-time p99*. Where the tested workload is read-simple, or for the tested MySQL single-row insert where shared commit behavior contributes materially under default durability, the observed layer spread is smaller and may be met by adding application processes until another bottleneck (contention, connection limits). These are performance findings only: the choice also turns on correctness, maintainability, security, type safety, and developer productivity, which this study does not measure and may outweigh them; and the common-SQL results are a compound sensitivity contrast on the selected-configuration cost, not a licence to use raw SQL.

6. Threats to Validity

We organize the threats along the four standard validity categories [62, 48].

Construct validity of the measures. Our dependent variables are HTTP-level throughput (req/s) and tail latency (p99) at the Express endpoint, not query-execution time at the driver or database boundary — deliberate, since it captures the full request round trip (Express routing, serialization), though the overhead we attribute to the *access layer* may partly reflect these surrounding costs. We hold those near-constant: every adapter satisfies a documented *adapter contract* and funnels rows through shared canonical constructors (fixed field set, key order, typing, under `TZ=UTC`). These constructors copy and type-normalize already-grouped rows or entity graphs; they do not query, join, regroup, sort, or cache. Their standalone median is 9.35–9.36 μs for the ten-comment deep fetch (0.072% of the fastest reported HTTP p50; see the supplement's canonical-constructor table), so it is too small to explain the multi-fold gap, while near-ties remain practically unresolved. The specification-derived

oracle replays the deterministic seed independently of all adapters and passes 73,080 expected-result checks. Differential checks then establish finite mutual equivalence. These checks found, among other defects, a Drizzle timestamp conversion that shifted PostgreSQL instants by the host UTC offset; it was fixed before measurement. The fixed-payload baseline ceiling sits above the fastest cell (Section 3), so the framework compresses rather than manufactures between-layer differences, attributable to how each layer reaches an identical response, not to what is returned. The reported single-row-insert finding uses each engine's *default* vendor-standard durability (Section 3), not one we relaxed, so those insert spreads (Table 5) describe the tested default deployment, not a tuning choice; a stack-specific relaxed-durability sensitivity quantifies how each configured treatment changes under that compound intervention without decomposing its cause (Section 5, Supplement Table S3).

Construct validity of the treatment. RQ1's treatment is each layer's *policy-selected documented* implementation and eager-loading strategy: an operational rule — the API the official documentation presents first — not validated against surveyed, typical, or performance-tuned usage. It is an intentionally artificial, predeclared selection policy, not a behavioral practitioner model. The frozen pages, conflict rule, and assignments make the decision auditable, but the single author both defined and applied it; two blind selection reviews are prepared and still pending. Where the policy differs from a performance-tuned configuration, RQ1's spread mixes the library's execution machinery with that strategy choice, made material by the one-to-four statement range on the deep fetch (Supplement Table S2). Three sensitivity checks qualify it. First, every selected path and alternative is preserved with the source-page hash (Supplement Table S33 and `METHODOLOGY.md`); this is provenance, not evidence of frequency or idiomaticity. Second, the *common-SQL raw-path sensitivity contrast* runs common SQL through every layer's raw facility, so the raw-path spread (deep fetch 1.71×, against 7.01× for the selected configuration; Section 4) is far smaller — a compound sensitivity contrast, not a decomposition

or bound on how much comes from the loading strategy versus the library machinery. Third, where the documented alternative is a byte-identical drop-in, a loading-strategy check (Supplement Table S18) shows the selected path is the *faster* strategy for the join-selected ORMs, so their deficit is not an artifact of a poor selected strategy; the lone exception, Objection's slower selected select-in path on MySQL, is disclosed rather than corrected. RQ1 therefore answers what this policy yields, not a library-only overhead.

Internal validity. Each adapter follows the registered policy and passes specification-derived and differential checks, but all were implemented by one author. A semantically correct yet needlessly allocating, converting, or serializing adapter would pass; no independent human implementation review is claimed. The result-blind author self-audit found two avoidable timed-path operations before accepting a campaign: Knex passed its PostgreSQL-style `returning` option on MySQL, producing 44,907 console warnings in fewer than two pilot repetitions, and MikroORM's aggregation obtained and discarded an unused underlying Knex handle once per request. Both were removed without changing the declared treatment or SQL; the affected pilots, logs, partial data, environments, and source manifests were preserved, the full admission chain passed again, and measurement restarted from repetition 1. A per-adapter coverage and disposition record is in `notes/adapter-self-audit.md`; because the implementer performed it, this remains author review and does not satisfy R7. Per-cell warm-up phases exceeding the slowest layer's cold-start stabilization absorb JIT, pool-fill, and plan-cache effects [63]; two further controls address non-layer confounds: one correlated-subquery aggregation plan, and the corrected-state campaign's verified reset before each write cell. The new state oracle discovered that the historical MySQL `AUTO_INCREMENT` reset could allocate below the cleanup floor, allowing old benchmark inserts to persist. Accordingly, historical MySQL range-scan, aggregation, and insert cells are not used as clean evidence; all five patterns are remeasured in one coherent campaign. Within a replicate every layer and engine runs the identical

PRNG-seeded id sequence, and a forward-versus-reversed write-cell-order check found no detectable order effect (Section 3), so order does not confound layer identity with run position. A residual risk is coordinated omission in the load generator, which can under-report tail latency at saturation [16, 55]; coordinated-omission-corrected constant-arrival sweeps on representative deep-fetch layers show small tails below measured capacity and sharp growth near saturation (Section 4; Supplement Tables S6 and S17).

External validity. All measurements come from a single schema, dataset size, and 16-vCPU virtualized host, a pool fixed at ten, and specific engine and library versions (PostgreSQL 18.4, MySQL 9.7.1; latest compatible stable versions as of the 15 July 2026 freeze, Supplement Table S11). The fixed pool does not drive the ranking: the pool-size frontier (Supplement Tables S7, S25) preserves the ordering from 1 to 50 connections, and a mixed read/write workload (S26) preserves the read ordering. Two choices are disclosed rather than resolved: Sequelize on its stable 6.x line (7.x is prerelease), and Prisma's MySQL path on the MariaDB adapter, as Prisma ships no `mysql2` adapter — an implementation×engine asymmetry. The memory-resident working set is a hot-cache regime that makes access-layer overhead more visible; as a workload becomes I/O-bound the spreads may compress toward 1×. Same-host checks do not bound an independent substrate: a post-restart rerun moved absolute throughput by up to 16%. The corrected-state 25-run campaign repeats all five patterns with a new randomized order and environment fingerprint on the same host. Cross-campaign agreement or disagreement is reported directly; it reduces dependence on one overnight ordering but does not address host, hypervisor-neighbor, CPU-steal, or frequency transfer. No empirical ordering is intended to travel beyond the tested campaigns, and close cross-backend rank reversals are exploratory unless their direction repeats. The protocol evidence is also bounded: its external audit has one rater, so general independent applicability remains to be tested.

Conclusion validity. Our analysis (Section 3) reports medians with the coefficient of variation and within-campaign bootstrap 95% intervals (repeatability, not population-general) and, because the design is a randomized block, compares adjacently ranked layers with *paired* tests on their per-replicate throughput ratios (paired sign-flip permutation, cross-checked with Wilcoxon; Supplement Table S36). Two bounds reinforce the ordering: the widest spreads dwarf the run-to-run CV ($\leq$ 4.5%; Supplement Tables S1, S9), and it holds across the concurrency sweep at $\geq$ 32 connections (Supplement Figure S2). Rather than assert a specific statistical power (absent a prespecified effect-size model), we rest the conclusions on effect sizes and interval estimates — foremost the *predefined* native-driver contrasts (Supplement Table S41), keeping the ranking-selected adjacent-pair permutation tests only as a descriptive post-hoc check. Application processes do not cross adapter, engine, or replicate blocks, although read endpoints within a block share one process [64]. All blocks share one host; primary intervals remain within-campaign, while the corrected RQ2 subset has a second same-host campaign. The secondary experiments covering only a layer subset or few replicates (open-loop tail, equal-CPU, pool-size, transactional write, and cluster checks; Supplement Table S32) are read as *sensitivity evidence* on the tested subset, not as claims about all eleven implementations.

7. Conclusion and Future Work

This paper proposes a concrete discipline for access-layer comparisons: derive expected read results from the declared task, check mutual equivalence and post-write state before timing, define each configured treatment by a predeclared policy, and report fixed-demand latency separately from capacity and matched utilization. The case-study policy selects documented configurations from frozen pages; it is reproducible evidence about those treatments, not observed practitioner behavior. The common-SQL raw-path experiment is a compound sensitivity contrast, not a library-intrinsic overhead estimate.

In the pinned Express.js case study, the largest observed gap occurs on the

tested deep fetch: 7.01× on PostgreSQL and 5.12× on MySQL, narrowing to 1.71× and 2.03× when the layers use common SQL through raw facilities. At stable 50% and most 70% capacity targets, p99 values form a much smaller band than at the high-load point. These are treatment- and campaign-specific observations; handwritten native SQL and ORM relation APIs are deliberately asymmetric practical implementations.

The evidence record separates completed checks from unperformed validation. The seed-derived oracle, state preflight, second same-host campaign, machine-readable checklist, and retrospective audit make the method inspectable. The external audit has one rater, every adapter remains single-author, and both campaigns use the same virtualized host. Result-blind review packets are therefore provided rather than an unsupported claim of independent implementation validation.

Future work should obtain independent treatment and adapter reviews, estimate inter-rater agreement when the protocol is applied to external benchmarks, repeat the reduced RQ2 matrix on a physically independent host, and extend the workload across datasets, topologies, and cold-cache regimes. The released harness and explicit compliance levels make those extensions additive rather than changes to an implied universal ranking.

CRediT authorship contribution statement

Mateusz Miotk: Conceptualization, Methodology, Software, Investigation, Data curation, Formal analysis, Visualization, Writing – original draft, Writing – review & editing.

Declaration of competing interest

The author declares no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper. The author is not affiliated with, nor funded by, any of the database libraries or vendors evaluated in this study.

Data availability

The public repository is available at `https://github.com/mmiotk/express-db-access-performance`; the archived published baseline is release v1.12.13 under the Zenodo concept DOI (`https://doi.org/10.5281/zenodo.21313858`) [65]. The present revision candidate adds the specification oracle, state preflight, corrected-state campaign, external-audit dataset, and review packets. A new immutable version DOI will be recorded before submission; until then these additions are not described as archived. A reproducibility summary — versions, requirements, the run commands, the time and resources needed, and which results the harness regenerates automatically from the archived raw data — is in Supplement Table S31 and `REPRODUCE.md`. **Exact reproduction requires the reference engines** (PostgreSQL 18.4 and MySQL 9.7.1), installed via the user-space conda path documented in `REPRODUCE.md`; the convenience Docker path pins the same engine versions but uses relaxed durability in a containerized topology and therefore reproduces the workflow, not the headline default-durability insert numbers. A machine-readable encoding of the protocol — inputs, mandatory and recommended stages, the cell-admission gate, and compliance levels — ships as `protocol-checklist.yaml` for auditing a benchmark against the protocol. The corrected campaign also records aggregate and per-file SHA-256 for the exact measurement source and schemas, supplementing the dirty-worktree commit identifier; exact 42-file source archives preserve and verify the measurement-time source state of both accepted campaigns. Reproduction status is separated explicitly. On 26 July 2026, an author-run isolated reconstruction of the current candidate verified 60/60 archived JSON hashes and regenerated 35 table outputs and four analysis JSON files byte-for-byte without starting either DBMS. An earlier author-run archive-isolated check of immutable v1.12.9 verified 35/35 inputs and reproduced 45/50 then-committed tables; its five presentation-only differences are identified in `notes/clean-room-reproduction.md`. Neither check is independent reproduction or a re-execution of the current measurements, and

neither is described as such.

Funding

This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

Declaration of generative AI and AI-assisted technologies in the writing process

During the preparation of this work the author used large language models (Claude, Anthropic, and Codex, OpenAI) to draft and edit manuscript text and improve its readability and language. After using these tools, the author reviewed and edited the content as needed and takes full responsibility for the content of the published article.

AI-assisted research software

Separately from the writing above, large language models (Claude, Anthropic, and Codex, OpenAI) also assisted the author in implementing and analyzing the benchmark harness. This is research tooling for which the author takes full responsibility, and the AI-assisted analytical code is publicly inspectable and checked: the statistical estimators carry unit tests against hand-computed values and known closed-form properties (`bench/stats.test.mjs`, run by `npm test`), every reported table cell traces to a raw per-cell record through a committed generator (`MANIFEST.md`), and the specification-derived read oracle checks expected results without using timed native-driver output; differential checks then establish finite cross-implementation equivalence; and write checks validate resulting database state before timing is trusted.

References

[1] T.-H. Chen, W. Shang, Z. M. Jiang, A. E. Hassan, M. Nasser, P. Flora, Detecting performance anti-patterns for applications developed using object-relational mapping, in: Proceedings of the 36th International Conference on Software Engineering (ICSE), ACM, 2014, pp. 1001–1012. doi:10.1145/2568225.2568259.

[2] M. Fowler, Patterns of Enterprise Application Architecture, Addison-Wesley, Boston, MA, 2002.

[3] I. K. Chaniotis, K.-I. D. Kyriakou, N. D. Tselikas, Is Node.js a viable option for building modern web applications? a performance evaluation study, Computing 97 (10) (2015) 1023–1044. doi:10.1007/s00607-014-0394-9.

[4] P. Kuffeł, B. Walter, Changes in Node.js server-side application performance characteristics over time under load, in: Advances in Information Systems Development, Vol. 77 of Lecture Notes in Information Systems and Organisation, Springer, 2025, pp. 275–294. doi:10.1007/978-3-031-87880-0_14.

[5] K. X. Carmo, F. Ferreira, E. Figueiredo, Performance evaluation of back-end frameworks: A comparative study, in: Proceedings of the 20th Brazilian Symposium on Information Systems (SBSI), ACM, 2024, pp. 1–9. doi:10.1145/3658271.3658314.

[6] D. Colley, C. Stanier, M. Asaduzzaman, The impact of object-relational mapping frameworks on relational query performance, in: 2018 International Conference on Computing, Electronics & Communications Engineering (iCCECE), IEEE, 2018, pp. 47–52, java, C#, Python, PHP — deliberately excludes JavaScript/Node. doi:10.1109/iccecome.2018.8659222.

[7] J. C. Yusmita, R. Arya, J. M. Wijaya, K. M. Suryaningrum, R. R. Siswanto, Optimizing database access strategy: A performance analysis comparison

of raw sql and prisma orm, Procedia Comput. Sci. 269 (2025) 1201–1210. doi:10.1016/j.procs.2025.09.061.

[8] J. Attala, C. Khemapatpapan, Comparing the performance of prisma, typeorm, and sequelize on postgresql, J. Comput. Creat. Technol. 3 (2) (2025) 201–213. doi:10.14456/jcct.2025.16.
URL https://so13.tci-thaijo.org/index.php/jcct/article/view/2330

[9] I. P. A. E. Pratama, I. M. S. Raharja, Node.js performance benchmarking and analysis at VirtualBox, Docker, and Podman environment using Node-Bench method, Int. J. Inform. Vis. 7 (4) (2023) 2240–2246. doi:10.62527/joiv.7.4.1762.

[10] S. Zhadko-Bazilevych, Analysis of ORM framework approaches for Node.js, J. Comput. Sci. Inst. 37 (2025) 426–430. doi:10.35784/jcsi.7951.

[11] S. V. Salunke, A. Ouda, A performance benchmark for the PostgreSQL and MySQL databases, Future Internet 16 (10) (2024) 382. doi:10.3390/fi16100382.

[12] Drizzle Team, Drizzle orm benchmarks, https://orm.drizzle.team/benchmarks, k6, 1M prepared requests, two-machine setup; reports req/s, avg + P95 latency, CPU (accessed 10 July 2026). (2024).

[13] M. D. Imam Sakti, D. Dirgantara, A. K. Ramadhan, M. A. Ibrahim, Optimizing asynchronous performance in Node.js with Express and PostgreSQL, Procedia Comput. Sci. 269 (2025) 172–181. doi:10.1016/j.procs.2025.08.270.

[14] M. Collina, et al., autocannon: fast http/1.1 benchmarking tool for node.js, https://github.com/mcollina/autocannon, version 7.15.0; accessed 10 July 2026. (2023).

[15] M. Raasveldt, P. Holanda, T. Gubner, H. Mühleisen, Fair benchmarking considered difficult: Common pitfalls in database performance testing,

in: Proceedings of the Workshop on Testing Database Systems (DBTest), ACM, 2018, pp. 1–6. doi:10.1145/3209950.3209955.

[16] M. Fruth, S. Scherzinger, W. Mauerer, R. Ramsauer, Tell-tale tail latencies: Pitfalls and perils in database benchmarking, in: Performance Evaluation and Benchmarking (TPCTC 2021), Vol. 13169 of Lecture Notes in Computer Science, Springer, 2022, pp. 119–134. doi:10.1007/978-3-030-94437-7_8.

[17] A. Georges, D. Buytaert, L. Eeckhout, Statistically rigorous Java performance evaluation, in: Proceedings of the 22nd ACM SIGPLAN Conference on Object-Oriented Programming Systems, Languages and Applications (OOPSLA), ACM, 2007, pp. 57–76. doi:10.1145/1297027.1297033.

[18] T. Sotiropoulos, S. Chaliasos, V. Atlidakis, D. Mitropoulos, D. Spinellis, Data-oriented differential testing of object-relational mapping systems, in: Proceedings of the 43rd IEEE/ACM International Conference on Software Engineering (ICSE), IEEE, 2021, pp. 1535–1547. doi:10.1109/ICSE43902.2021.00137.

[19] C. Ireland, D. Bowers, M. Newton, K. Waugh, A classification of object-relational impedance mismatch, in: 2009 First International Conference on Advances in Databases, Knowledge, and Data Applications (DBKDA), IEEE, 2009, pp. 36–43. doi:10.1109/DBKDA.2009.11.

[20] A. Torres, R. Galante, M. S. Pimenta, A. J. B. Martins, Twenty years of object-relational mapping: A survey on patterns, solutions, and their implications on application design, Inf. Softw. Technol. 82 (2017) 1–18. doi:10.1016/j.infsof.2016.09.009.

[21] P. J. Sadalage, M. Fowler, NoSQL Distilled: A Brief Guide to the Emerging World of Polyglot Persistence, Addison-Wesley, Upper Saddle River, NJ, 2012.

[22] J. Yang, P. Subramaniam, S. Lu, C. Yan, A. Cheung, How not to structure your database-backed web applications: A study of performance bugs in the wild, in: Proceedings of the 40th International Conference on Software Engineering (ICSE), ACM, 2018, pp. 800–810. doi:10.1145/3180155.3180194.

[23] C. Yan, A. Cheung, J. Yang, S. Lu, Understanding database performance inefficiencies in real-world web applications, in: Proceedings of the 2017 ACM Conference on Information and Knowledge Management (CIKM), ACM, 2017, pp. 1299–1308. doi:10.1145/3132847.3132954.

[24] S. Shao, Z. Qiu, X. Yu, W. Yang, G. Jin, T. Xie, X. Wu, Database-access performance antipatterns in database-backed web applications, in: 2020 IEEE International Conference on Software Maintenance and Evolution (ICSME), IEEE, 2020, pp. 58–69. doi:10.1109/ICSME46990.2020.00016.

[25] T.-H. Chen, W. Shang, Z. M. Jiang, A. E. Hassan, M. Nasser, P. Flora, Finding and evaluating the performance impact of redundant data access for applications that are developed using object-relational mapping frameworks, IEEE Trans. Softw. Eng. 42 (12) (2016) 1148–1161. doi:10.1109/TSE.2016.2553039.

[26] A. Cheung, A. Solar-Lezama, S. Madden, Optimizing database-backed applications with query synthesis, in: Proceedings of the 34th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI), ACM, 2013, pp. 3–14. doi:10.1145/2491956.2462180.

[27] A. Cheung, S. Madden, A. Solar-Lezama, Sloth: Being lazy is a virtue (when issuing database queries), in: Proceedings of the 2014 ACM SIGMOD International Conference on Management of Data, ACM, 2014, pp. 931–942. doi:10.1145/2588555.2593672.

[28] T.-H. Chen, W. Shang, A. E. Hassan, M. Nasser, P. Flora, CacheOptimizer: Helping developers configure caching frameworks for Hibernate-based database-centric web applications, in: Proceedings of the 2016 24th

ACM SIGSOFT International Symposium on Foundations of Software Engineering (FSE), ACM, 2016, pp. 666–677. doi:10.1145/2950290.2950303.

[29] A. Turcotte, M. D. Shah, M. W. Aldrich, F. Tip, DrAsync: Identifying and visualizing anti-patterns in asynchronous JavaScript, in: Proceedings of the 44th International Conference on Software Engineering (ICSE), ACM, 2022, pp. 774–785. doi:10.1145/3510003.3510097.

[30] M. Lorenz, J.-P. Rudolph, G. Hesse, M. Uflacker, H. Plattner, Object-relational mapping revisited—a quantitative study on the impact of database technology on O/R mapping strategies, in: Proceedings of the 50th Hawaii International Conference on System Sciences (HICSS), 2017. doi:10.24251/HICSS.2017.592.

[31] P. van Zyl, D. G. Kourie, L. Coetzee, A. Boake, The influence of optimisations on the performance of an object relational mapping tool, in: Proceedings of the 2009 Annual Research Conference of the South African Institute of Computer Scientists and Information Technologists (SAICSIT), ACM, 2009, pp. 150–159. doi:10.1145/1632149.1632169.

[32] A. Bonvoisin, C. Quinton, R. Rouvoy, Understanding the performance-energy tradeoffs of object-relational mapping frameworks, in: 2024 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER), IEEE, 2024, pp. 626–636. doi:10.1109/SANER60148.2024.00069.

[33] Y. Li, S. Manoharan, A performance comparison of SQL and NoSQL databases, in: 2013 IEEE Pacific Rim Conference on Communications, Computers and Signal Processing (PACRIM), IEEE, 2013, pp. 15–19. doi:10.1109/PACRIM.2013.6625441.

[34] B. F. Cooper, A. Silberstein, E. Tam, R. Ramakrishnan, R. Sears, Benchmarking cloud serving systems with YCSB, in: Proceedings of the 1st ACM Symposium on Cloud Computing (SoCC), ACM, 2010, pp. 143–154. doi:10.1145/1807128.1807152.

[35] J. Dean, L. A. Barroso, The tail at scale, Commun. ACM 56 (2) (2013) 74–80. doi:10.1145/2408776.2408794.

[36] J. Li, N. K. Sharma, D. R. K. Ports, S. D. Gribble, Tales of the tail: Hardware, OS, and application-level sources of tail latency, in: Proceedings of the ACM Symposium on Cloud Computing (SoCC), ACM, 2014, pp. 1–14. doi:10.1145/2670979.2670988.

[37] Z. M. Jiang, A. E. Hassan, A survey on load testing of large-scale software systems, IEEE Trans. Softw. Eng. 41 (11) (2015) 1091–1118. doi:10.1109/TSE.2015.2445340.

[38] P. Leitner, C.-P. Bezemer, An exploratory study of the state of practice of performance testing in Java-based open source projects, in: Proceedings of the 8th ACM/SPEC International Conference on Performance Engineering (ICPE), ACM, 2017, pp. 373–384. doi:10.1145/3030207.3030213.

[39] C. Collberg, T. A. Proebsting, Repeatability in computer systems research, Commun. ACM 59 (3) (2016) 62–69. doi:10.1145/2812803.

[40] K. Huppler, The art of building a good benchmark, in: Performance Evaluation and Benchmarking (TPCTC 2009), Vol. 5895 of Lecture Notes in Computer Science, Springer, 2009, pp. 18–30. doi:10.1007/978-3-642-10424-4_3.

[41] S. E. Sim, S. Easterbrook, R. C. Holt, Using benchmarking to advance research: A challenge to software engineering, in: Proceedings of the 25th International Conference on Software Engineering (ICSE), IEEE, 2003, pp. 74–83. doi:10.1109/ICSE.2003.1201189.

[42] Prisma, Query performance benchmarks, `https://benchmarks.prisma.io/`, prisma/Drizzle/TypeORM; median of 500 iterations; no throughput, no p95/p99 (accessed 10 July 2026). (2024).

[43] Gel Data (EdgeDB), Imdbench: Orm benchmarks, `https://github.com/geldata/imdbench`, accessed 10 July 2026. (2024).

[44] S. Kounev, K.-D. Lange, J. von Kistowski, Systems Benchmarking: For Scientists and Engineers, Springer, 2020. doi:10.1007/978-3-030-41705-5.

[45] W. Hasselbring, Benchmarking as empirical standard in software engineering research, in: Proceedings of the Evaluation and Assessment in Software Engineering (EASE), ACM, 2021, pp. 457–462. doi:10.1145/3463274.3463361.

[46] M. Rigger, Z. Su, Testing database engines via pivoted query synthesis, in: Proceedings of the 14th USENIX Symposium on Operating Systems Design and Implementation (OSDI), USENIX Association, 2020, pp. 667–682.

[47] V. R. Basili, H. D. Rombach, The TAME project: Towards improvement-oriented software environments, IEEE Trans. Softw. Eng. 14 (6) (1988) 758–773. doi:10.1109/32.6156.

[48] C. Wohlin, P. Runeson, M. Höst, M. C. Ohlsson, B. Regnell, A. Wesslén, Experimentation in Software Engineering, 2nd Edition, Springer, Berlin, Heidelberg, 2012. doi:10.1007/978-3-642-29044-2.

[49] B. A. Kitchenham, S. L. Pfleeger, L. M. Pickard, P. W. Jones, D. C. Hoaglin, K. El Emam, J. Rosenberg, Preliminary guidelines for empirical research in software engineering, IEEE Trans. Softw. Eng. 28 (8) (2002) 721–734. doi:10.1109/TSE.2002.1027796.

[50] S. Harizopoulos, D. J. Abadi, S. Madden, M. Stonebraker, OLTP through the looking glass, and what we found there, in: Proceedings of the 2008 ACM SIGMOD International Conference on Management of Data, ACM, 2008, pp. 981–992. doi:10.1145/1376616.1376713.

[51] K. Gupta, M. Mathuria, Improving performance of web application approaches using connection pooling, in: 2017 International Conference of Electronics, Communication and Aerospace Technology (ICECA), IEEE, 2017, pp. 355–358. doi:10.1109/ICECA.2017.8212833.

[52] R. Laigner, Y. Zhou, M. A. Vaz Salles, Y. Liu, M. Kalinowski, Data management in microservices: State of the practice, challenges, and research directions, Proc. VLDB Endow. 14 (13) (2021) 3348–3361. doi:10.14778/3484224.3484232.

[53] B. Schroeder, A. Wierman, M. Harchol-Balter, Open versus closed: A cautionary tale, in: Proceedings of the 3rd USENIX Symposium on Networked Systems Design and Implementation (NSDI), USENIX Association, 2006. URL https://www.usenix.org/legacy/event/nsdi06/tech/schroeder.html

[54] X. Yu, G. Bezerra, A. Pavlo, S. Devadas, M. Stonebraker, Staring into the abyss: An evaluation of concurrency control with one thousand cores, Proc. VLDB Endow. 8 (3) (2014) 209–220. doi:10.14778/2735508.2735511.

[55] G. Tene, How NOT to measure latency, https://www.infoq.com/presentations/latency-response-time/, conference presentation; origin of the term "coordinated omission" (accessed 10 July 2026). (2015).

[56] B. Krishnamachari, How to do statistical evaluations in ECE/CS papers: A practical playbook for defensible results, https://arxiv.org/abs/2605.00428 (2026).

[57] C. Laaber, J. Scheuner, P. Leitner, Software microbenchmarking in the cloud. how bad is it really?, Empir. Softw. Eng. 24 (4) (2019) 2469–2508. doi:10.1007/s10664-019-09681-1.

[58] A. Arcuri, L. Briand, A hitchhiker's guide to statistical tests for assessing randomized algorithms in software engineering, Softw. Test. Verif. Reliab. 24 (3) (2014) 219–250. doi:10.1002/stvr.1486.

[59] N. Cliff, Dominance statistics: Ordinal analyses to answer ordinal questions, Psychol. Bull. 114 (3) (1993) 494–509. doi:10.1037/0033-2909.114.3.494.

[60] A. V. Papadopoulos, L. Versluis, A. Bauer, N. Herbst, J. von Kistowski, A. Ali-Eldin, C. L. Abad, J. N. Amaral, P. Tůma, A. Io-

sup, Methodological principles for reproducible performance evaluation in cloud computing, IEEE Trans. Softw. Eng. 47 (8) (2021) 1528–1543. doi:10.1109/TSE.2019.2927908.

[61] V. Leis, A. Gubichev, A. Mirchev, P. Boncz, A. Kemper, T. Neumann, How good are query optimizers, really?, Proc. VLDB Endow. 9 (3) (2015) 204–215. doi:10.14778/2850583.2850594.

[62] T. D. Cook, D. T. Campbell, Quasi-Experimentation: Design and Analysis Issues for Field Settings, Houghton Mifflin, Boston, MA, 1979.

[63] T. Mytkowicz, A. Diwan, M. Hauswirth, P. F. Sweeney, Producing wrong data without doing anything obviously wrong!, in: Proceedings of the 14th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS), ACM, 2009, pp. 265–276. doi:10.1145/1508244.1508275.

[64] T. Kalibera, R. Jones, Rigorous benchmarking in reasonable time, in: Proceedings of the 2013 International Symposium on Memory Management (ISMM), ACM, 2013, pp. 63–74. doi:10.1145/2464157.2464160.

[65] M. Miotk, Replication package for A Comparability Protocol for Benchmarking Relational Database Access Layers in Express.js, Zenodo, version 1.12.13, [software and dataset] (2026). doi:10.5281/zenodo.21313858.